

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250327>

Outline : Convolutional Neural Networks (code [samples](#))

- Definitions / Motivation
- The Architecture of the Visual Cortex
- Convolutional Layer
- Pooling Layer
- CNN Architectures
- Object Detection
- Semantic Segmentation
- Takeaways

—

Source: Aurelien Geron, "Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems", 2019 ([Available online](#)) / **Chapter 14**

Definition / Motivation

Perception largely takes place outside the realm of our consciousness: **specialized** visual, auditory, and other sensory modules in our brains: our consciousness gets “high-level features”.

Convolutional neural networks (CNNs) emerged from the study of the brain’s visual cortex.

CNNs have managed to achieve superhuman performance on some complex visual tasks: image search services, self-driving cars, automatic video classification systems, and more. Also good at as *voice recognition* or *natural language processing* (NLP).

The Architecture of the Visual Cortex

- The higher-level neurons are based on the outputs of neighboring lower-level neurons.
- This is a powerful architecture able to detect all sorts of complex patterns in any area of the visual field.

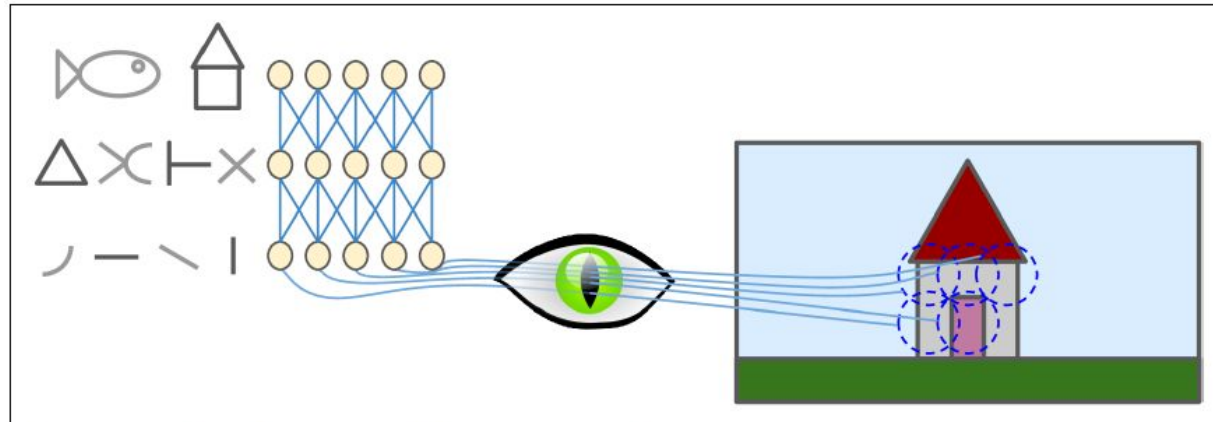


Figure 14-1. Local receptive fields in the visual cortex

Stacking Multiple Feature Maps

2D $\rightarrow$ 3D: a convolutional layer has multiple filters (you decide how many), and it outputs one feature map per filter, so it is more accurately represented in 3D.

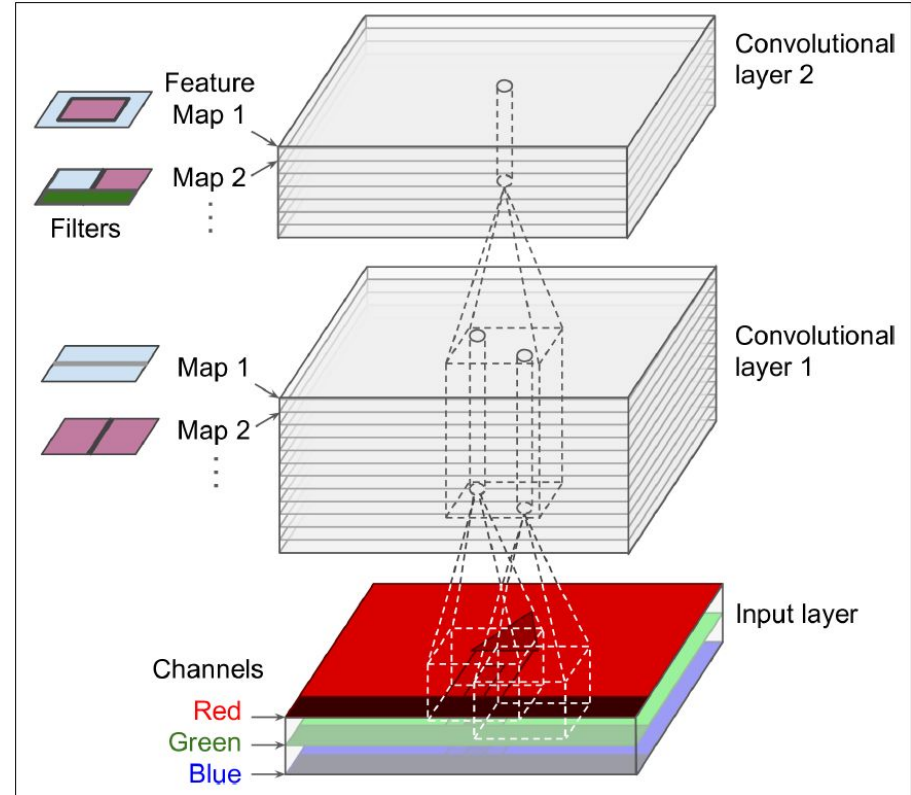


Figure 14-6. Convolution layers with multiple feature maps, and images with three color channels

Pooling Layer (1) Steps 18-29

- Now, the goal is to subsample (i.e., shrink) the input image in order to reduce the computational load, the memory usage, and the number of parameters (thereby limiting the risk of overfitting).
- Each neuron in a pooling layer is connected to the outputs of a limited number of neurons in the previous layer, located within a small rectangular receptive field.
- Define its size, the stride, and the padding type, as before.
- A **pooling neuron has no weights**; aggregates the inputs using an **aggregation function such as the max or mean**.

Pooling Layer (2)

2×2 pooling kernel (just a stateless sliding window), with a stride of 2, and no padding. **Only the max input value in each receptive field makes it to the next layer**, while the other inputs are dropped.

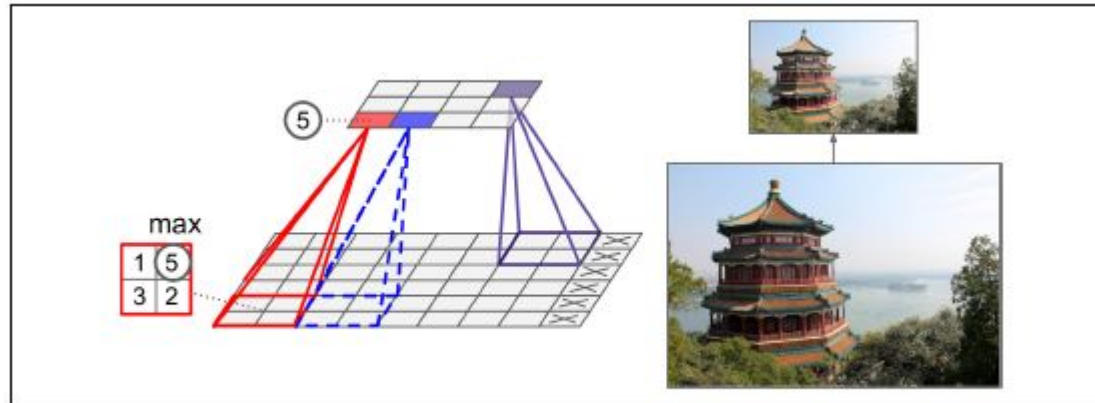


Figure 14-8. Max pooling layer (2×2 pooling kernel, stride 2, no padding)

Pooling Layer (3)

Pooling layer also introduces some level of invariance to small translations.
 2×2 kernel and stride 2. → Translation invariance: A and B are identical. C: 50% invariance.

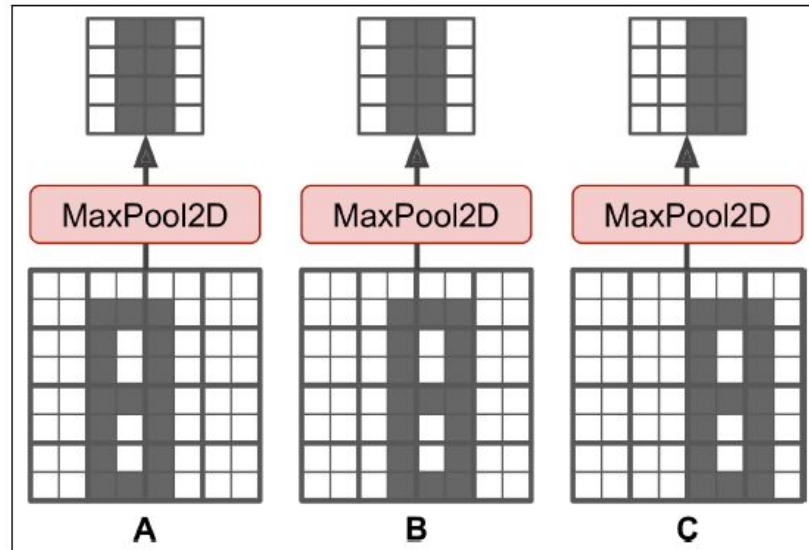


Figure 14-9. Invariance to small translations

Pooling Layer (4)

- By inserting a max pooling layer every few layers in a CNN, it is possible to get some level of translation invariance at a larger scale.
- Also offers a small amount of rotational invariance and a slight scale invariance, which can be useful in cases where the prediction should not depend on these details, such as in classification tasks.
- Downsides: very destructive, for example, with a tiny 2×2 kernel and a stride of 2, the output will be two times smaller in both directions (so its area will be four times smaller), simply dropping 75% of the input values.
- Not desired for certain applications like *semantic segmentation*.

Tensorflow implementation (1)

MaxPool2D, AvgPooling2D

To create a max pooling layer using a 2×2 kernel. The strides default to the kernel size, so this layer will use a stride of 2 (both horizontally and vertically). By default, it uses VALID padding (i.e., no padding at all):

```
max_pool = keras.layers.MaxPool2D(pool_size=2)
```

For *average pooling layer* - AvgPooling2D

Tensorflow implementation (2)

MaxPool2D, AvgPooling2D

AvgPooling2D computes *mean* rather than *max*.

- Max is mostly used. Generally performs better (counter-intuitively, as mean loses less information).
- But max pooling preserves only the “strongest” features, getting rid of all the “meaningless” ones.
- Max pooling offers stronger translation invariance than average pooling, as well as slightly less computations.

Tensorflow implementation (3)

Max pooling and average pooling can be performed along the depth dimension rather than the spatial dimensions. This can allow the CNN to learn to be invariant to various features.

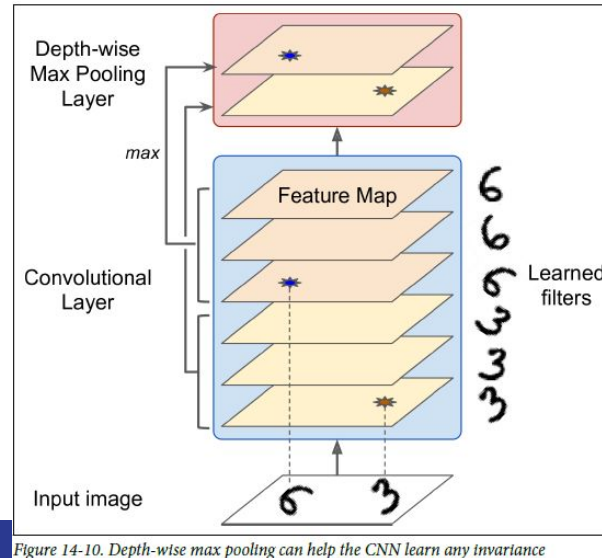


Figure 14-10. Depth-wise max pooling can help the CNN learn any invariance

Tensorflow implementation (4)

To learn multiple filters, each detecting a different rotation of the same pattern, such as handwritten digits, and the depth-wise max pooling layer would ensure that the output is the same regardless of the rotation. The CNN could similarly learn to be invariant to anything else: thickness, brightness, skew, color, and so on.

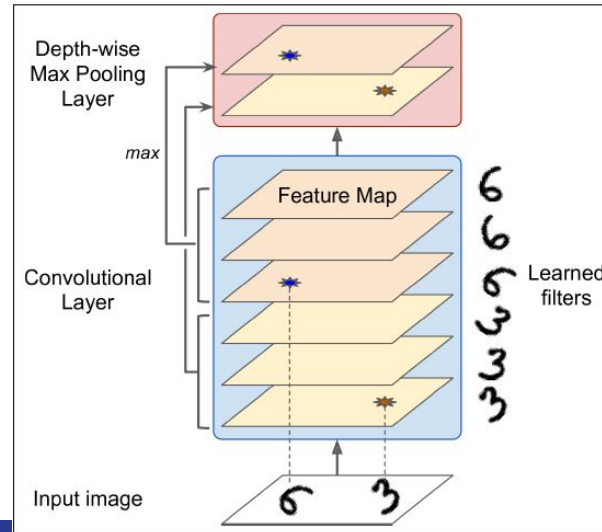


Figure 14-10. Depth-wise max pooling can help the CNN learn any invariance

Tensorflow implementation (5)

[tf.nn.max_pool\(\)](#)

The first three values of each should be 1: this indicates that the kernel size and stride along the batch, height and width dimensions should be 1. The last value should be whatever kernel size and stride you want along the depth dimension, for example 3 (must be a divisor of the input depth):

```
output = tf.nn.max_pool(images,  
                          ksize=(1, 1, 1, 3),  
                          strides=(1, 1, 1, 3),  
                          padding="VALID")
```

Tensorflow implementation (6)

Global average pooling layer: computes the **mean of each entire feature map** (it's like an average pooling layer using a **pooling kernel with the same spatial dimensions as the inputs**). Outputs a single number per feature map and per instance.

[GlobalAveragePooling2D](#).

```
global_avg_pool = keras.layers.GlobalAvgPool2D()
```

CNN Architectures

LeNet-5

AlexNet

GoogLeNet

VGGNet

ResNet

Xception

SENet

Typical CNN (1)

Typical CNN: stack of a few convolutional layers (each one generally followed by a ReLU layer), then a pooling layer, then another few convolutional layers (+ReLU), then another pooling layer, and so on.

The image gets smaller and smaller as it progresses through the network, but it also typically gets deeper and deeper (i.e., with more feature maps) thanks to the convolutional layers.

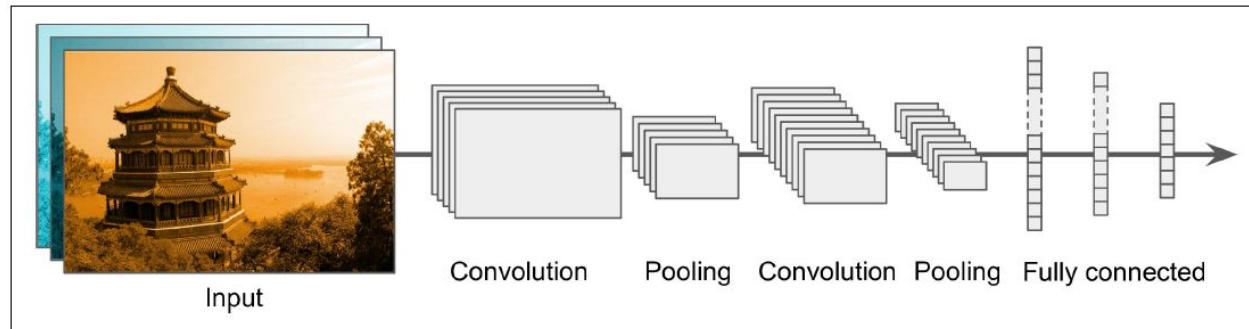


Figure 14-11. Typical CNN architecture

Typical CNN (2)

For fashion
MNIST:

```
from functools import partial

DefaultConv2D = partial(keras.layers.Conv2D,
                        kernel_size=3, activation='relu', padding="SAME")

model = keras.models.Sequential([
    DefaultConv2D(filters=64, kernel_size=7, input_shape=[28, 28, 1]),
    keras.layers.MaxPooling2D(pool_size=2),
    DefaultConv2D(filters=128),
    DefaultConv2D(filters=128),
    keras.layers.MaxPooling2D(pool_size=2),
    DefaultConv2D(filters=256),
    DefaultConv2D(filters=256),
    keras.layers.MaxPooling2D(pool_size=2),
    keras.layers.Flatten(),
    keras.layers.Dense(units=128, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=64, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=10, activation='softmax'),
])
```

Typical CNN (3)

- In this code, we start by using the `partial()` function to define a thin wrapper around the `Conv2D` class, called DefaultConv2D: avoids having to repeat the same hyperparameter values over and over again.
- The first layer uses a large kernel size, but no stride because the input images are not very large. It also sets `input_shape=[28, 28, 1]`, which means the images are **28 × 28 pixels**, with a **single color channel** (i.e., grayscale).
- Next, we have a max pooling layer, which divides each spatial dimension by a factor of two (since `pool_size=2`).

```
from functools import partial

DefaultConv2D = partial(keras.layers.Conv2D,
                        kernel_size=3, activation='relu', padding="SAME")

model = keras.models.Sequential([
    DefaultConv2D(filters=64, kernel_size=7, input_shape=[28, 28, 1]),
    keras.layers.MaxPooling2D(pool_size=2),
    DefaultConv2D(filters=128),
    DefaultConv2D(filters=128),
    keras.layers.MaxPooling2D(pool_size=2),
    DefaultConv2D(filters=256),
    DefaultConv2D(filters=256),
    keras.layers.MaxPooling2D(pool_size=2),
    keras.layers.Flatten(),
    keras.layers.Dense(units=128, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=64, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=10, activation='softmax'),
])
```

Typical CNN (4)

- Then the same structure is repeated twice: two convolutional layers followed by a max pooling layer.
For larger images, we could repeat this structure several times (the number of repetitions is a hyperparameter can be tuned).
- The number of filters grows as we climb up the CNN towards the output layer (it is initially **64**, then **128**, then **256**) → the number of low level features is often fairly low (e.g., small circles, horizontal lines, etc.), but there are many different ways to combine them into higher level features.
- A common practice to double the number of filters after each pooling layer: since a pooling layer divides each spatial dimension by a factor of 2, we can afford doubling the number of feature maps in the next layer, without fear of exploding the number of parameters, memory usage, or computational load.

```
from functools import partial

DefaultConv2D = partial(keras.layers.Conv2D,
                        kernel_size=3, activation='relu', padding='SAME')

model = keras.models.Sequential([
    DefaultConv2D(filters=64, kernel_size=7, input_shape=[28, 28, 1]),
    keras.layers.MaxPooling2D(pool_size=2),
    DefaultConv2D(filters=128),
    DefaultConv2D(filters=128),
    keras.layers.MaxPooling2D(pool_size=2),
    DefaultConv2D(filters=256),
    DefaultConv2D(filters=256),
    keras.layers.MaxPooling2D(pool_size=2),
    keras.layers.Flatten(),
    keras.layers.Dense(units=128, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=64, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=10, activation='softmax'),
])
```

Typical CNN (5)

- Next is the fully connected network, composed of 2 hidden dense layers and a dense output layer.

Note that we must flatten its inputs, since a dense network expects a 1D array of features for each instance. We also add two dropout layers, with a dropout rate of 50% each, to reduce overfitting.

- Reaches over 92% accuracy on the test set. Not the best possible, but better than previous MLP seen before.
- ImageNet Large Scale Visual Recognition Challenge (ILSVRC) used to derive best performing CNN architectures.

```
from functools import partial
DefaultConv2D = partial(Keras.layers.Conv2D,
                        kernel_size=3, activation='relu', padding="SAME")

model = keras.models.Sequential([
    DefaultConv2D(filters=64, kernel_size=7, input_shape=[28, 28, 1]),
    keras.layers.MaxPooling2D(pool_size=2),
    DefaultConv2D(filters=128),
    keras.layers.MaxPooling2D(pool_size=2),
    DefaultConv2D(filters=256),
    keras.layers.MaxPooling2D(pool_size=2),
    keras.layers.Flatten(),
    keras.layers.Dense(units=128, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=64, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=10, activation='softmax'),
])
```

LeNet-5 architecture (1)

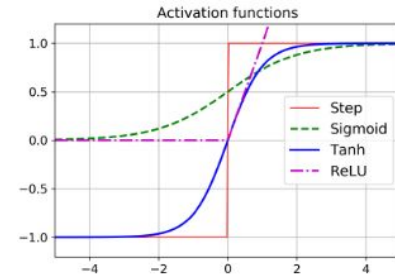


Table 14-1. LeNet-5 architecture

Layer	Type	Maps	Size	Kernel size	Stride	Activation
Out	Fully Connected	—	10	—	—	RBF
F6	Fully Connected	—	84	—	—	tanh
C5	Convolution	120	1×1	5×5	1	tanh
S4	Avg Pooling	16	5×5	2×2	2	tanh
C3	Convolution	16	10×10	5×5	1	tanh
S2	Avg Pooling	6	14×14	2×2	2	tanh
C1	Convolution	6	28×28	5×5	1	tanh
In	Input	1	32×32	—	—	—

LeNet-5 architecture (2)

- MNIST images are 28×28 pixels, but they are zero-padded to 32×32 pixels and normalized before being fed to the network. The rest of the network does not use any padding, which is why the size keeps shrinking as the image progresses through the network.
- The average pooling layers are slightly more complex than usual: each neuron computes the mean of its inputs, then multiplies the result by a learnable coefficient (one per map) and adds a learnable bias term (again, one per map), then finally applies the activation function.
- Most neurons in C3 maps are connected to neurons in only three or four S2 maps (instead of all six S2 maps). See [table 1 \(page 8\)](#) in the original paper¹⁰ for details.

LeNet-5 architecture (3)

- The output layer is a bit special: instead of computing the matrix multiplication of the inputs and the weight vector, each neuron outputs the square of the Euclidian distance between its input vector and its weight vector.
- Each output measures how much the image belongs to a particular digit class. The cross entropy cost function is now preferred, as it penalizes bad predictions much more, producing larger gradients and converging faster.

AlexNet CNN architecture (1) : 2012

Table 14-2. AlexNet architecture

Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully Connected	–	1,000	–	–	–	Softmax
F9	Fully Connected	–	4,096	–	–	–	ReLU
F8	Fully Connected	–	4,096	–	–	–	ReLU
C7	Convolution	256	13×13	3×3	1	SAME	ReLU
C6	Convolution	384	13×13	3×3	1	SAME	ReLU
C5	Convolution	384	13×13	3×3	1	SAME	ReLU
S4	Max Pooling	256	13×13	3×3	2	VALID	–
C3	Convolution	256	27×27	5×5	1	SAME	ReLU
S2	Max Pooling	96	27×27	3×3	2	VALID	–
C1	Convolution	96	55×55	11×11	4	VALID	ReLU
In	Input	3 (RGB)	227×227	–	–	–	–

AlexNet CNN architecture (2)

- Two regularization techniques:
 - dropout with a 50% dropout rate during training to the outputs of layers F8 and F9.
 - data augmentation by **randomly shifting the training images by various offsets, flipping them horizontally, and changing the lighting conditions.**

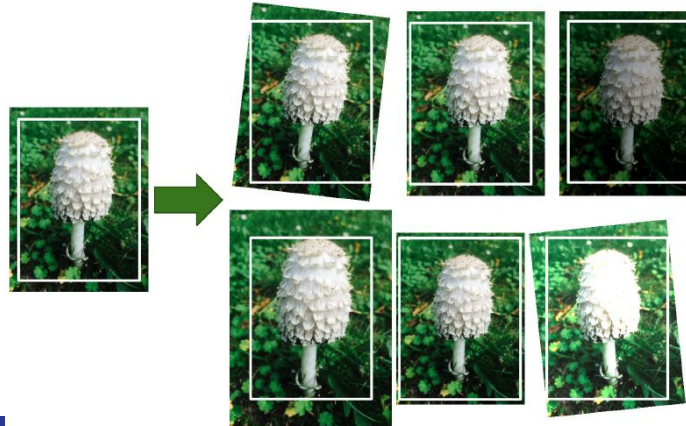


Figure 14-12. Generating new training instances from existing ones

GoogLeNet (1) 2014

Depth concat layer (stack the feature maps from all four top convolutional layers).

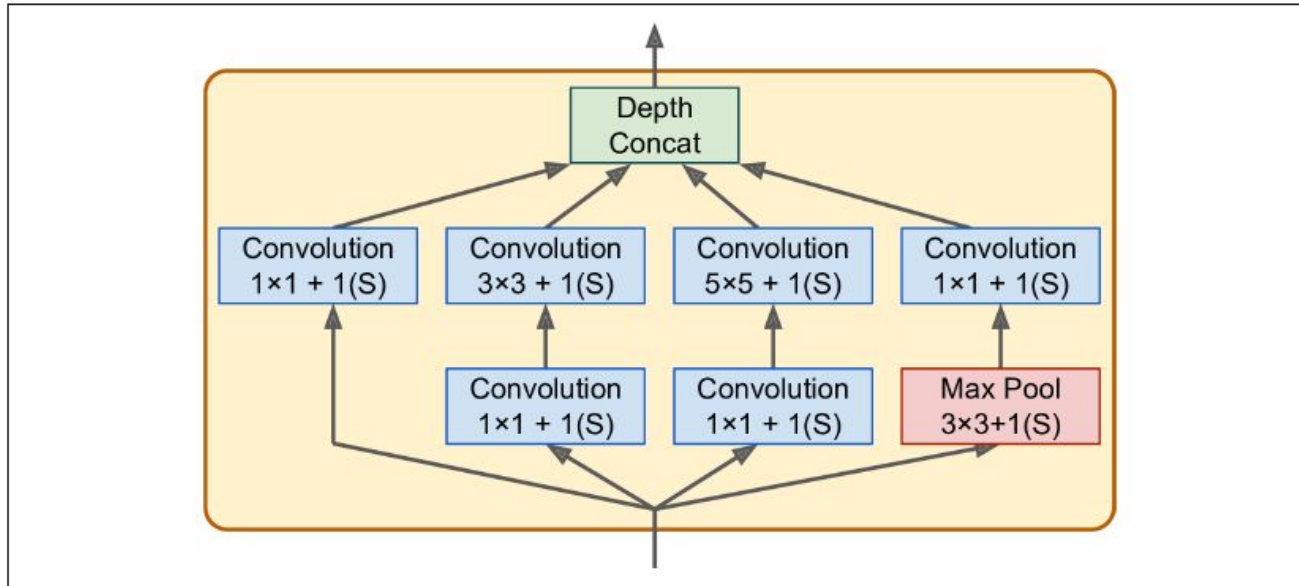


Figure 14-13. Inception module

GoogLeNet (2)

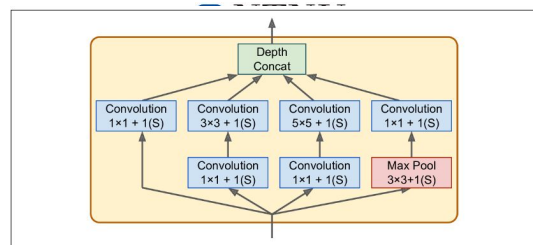


Figure 14-13. Inception module

Why inception modules have convolutional layers with 1×1 kernels?

- They cannot capture spatial patterns, they can **capture patterns along the depth dimension**.
- Output fewer feature maps than their inputs, so they serve as *bottleneck layers*, meaning they reduce dimensionality. → cuts the computational cost and the number of parameters, speeding up training and improving generalization.
- Each pair of convolutional layers ($[1 \times 1, 3 \times 3]$ and $[1 \times 1, 5 \times 5]$) acts like a single, powerful convolutional layer, capable of capturing more complex patterns.

GoogLeNet (3)

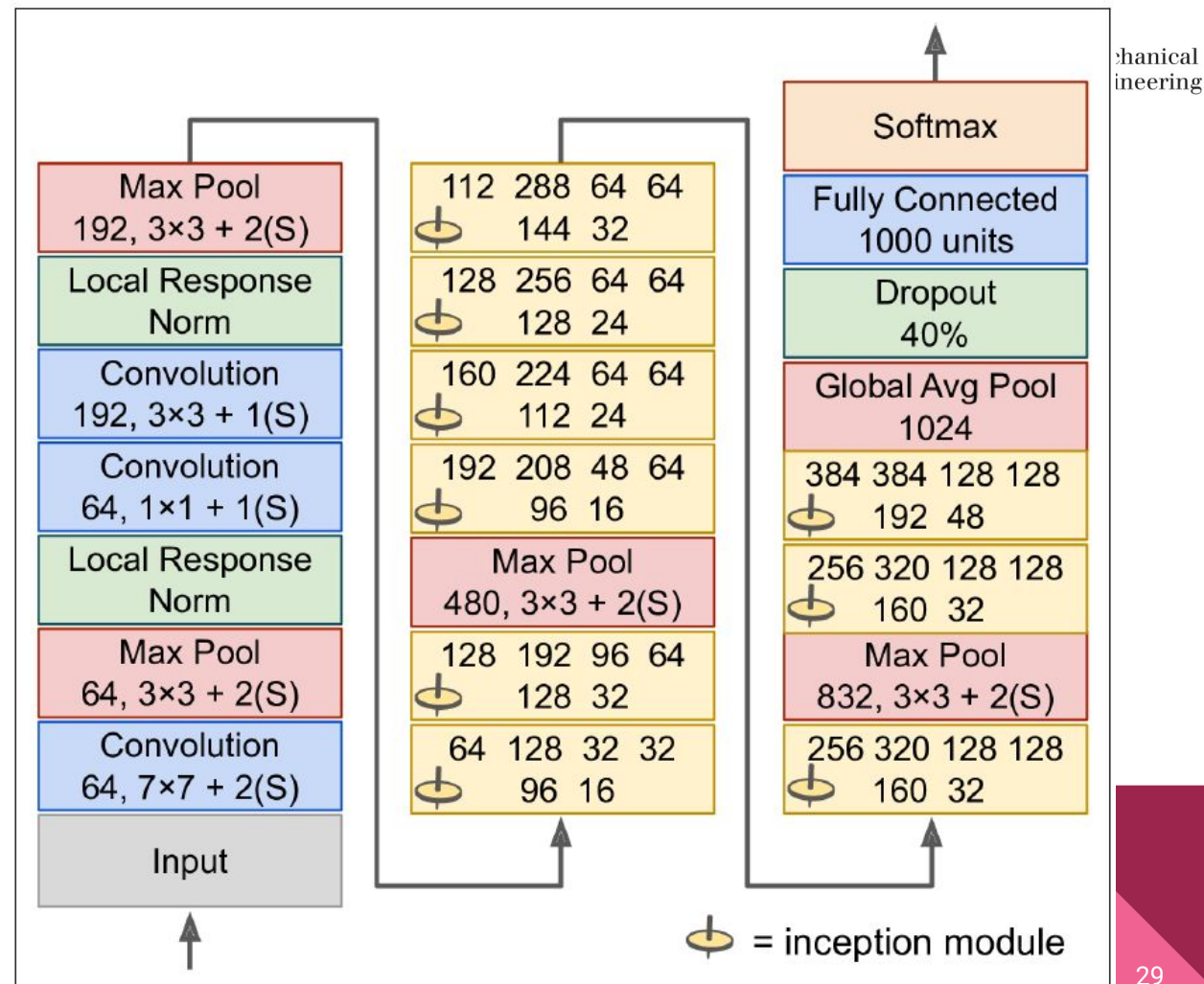


Figure 14-14. GoogLeNet architecture

VGGNet (1) : 2014

- Simple and classical architecture, with 2 or 3 convolutional layers, a pooling layer.
- Then again 2 or 3 convolutional layers, a pooling layer, and so on (with a total of just 16 convolutional layers).
- A final dense network with 2 hidden layers and the output layer. It used only 3×3 filters, but many filters.

ResNet (1) : 2015

Residual learning: The goal is to make it model a target function $h(\mathbf{x})$. If you add the input $\mathbf{x}$ to the output of the network (i.e., you add a skip connection), then the network will be forced to model $f(\mathbf{x}) = h(\mathbf{x}) - \mathbf{x}$ rather than $h(\mathbf{x})$.

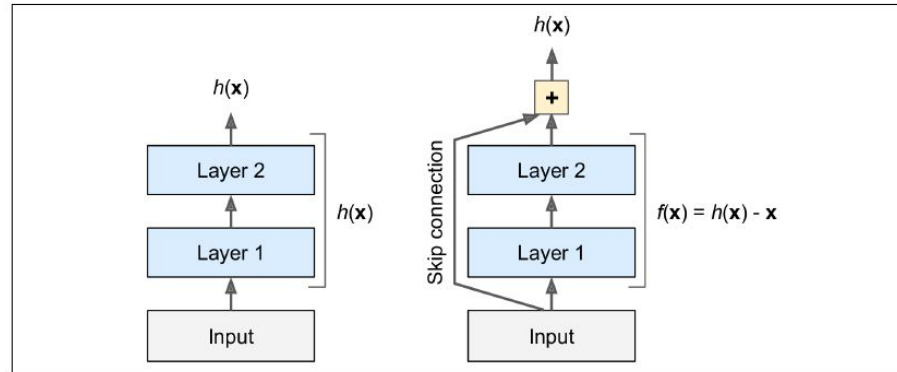


Figure 14-15. Residual learning

ResNet (2)

Error rate under 3.6%, using an extremely deep CNN composed of 152 layers. General trend: models are getting deeper and deeper, with fewer and fewer parameters to be able to train such a deep network is to use skip connections (also called shortcut connections). The signal feeding into a layer is also added to the output of a layer located a bit higher up the stack. Speeds up training.

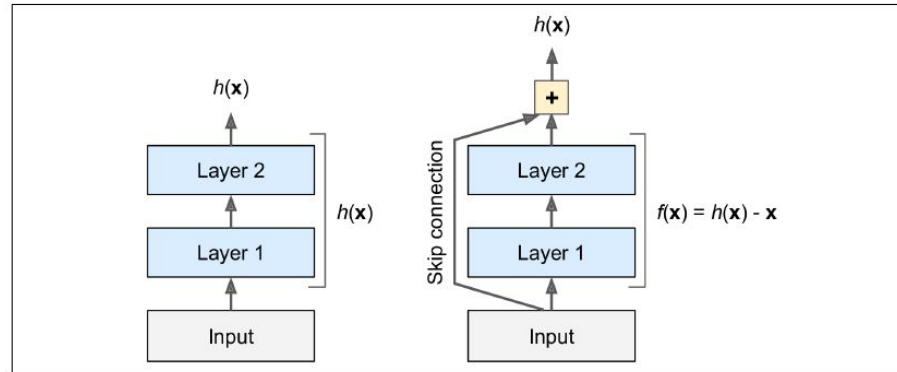


Figure 14-15. Residual learning

ResNet (3)

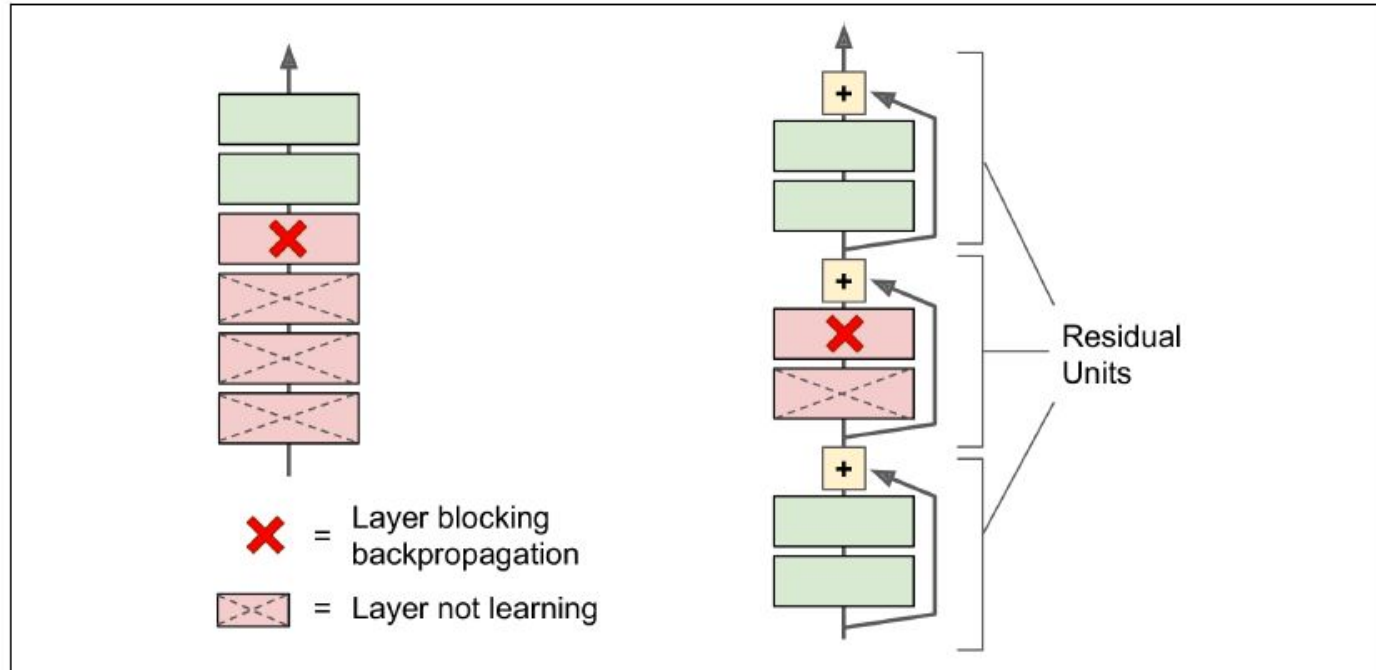


Figure 14-16. Regular deep neural network (left) and deep residual network (right)

ResNet (4)

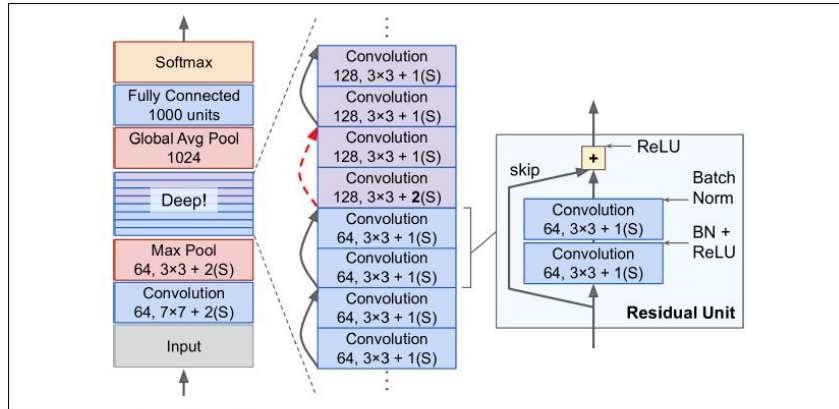


Figure 14-17. ResNet architecture

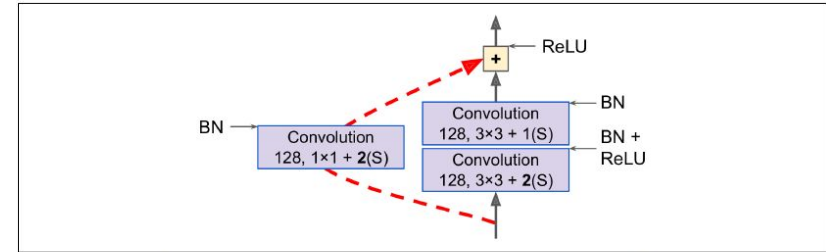


Figure 14-18. Skip connection when changing feature map size and depth

Xception : 2016

Error rate below 3% for image classification challenge.

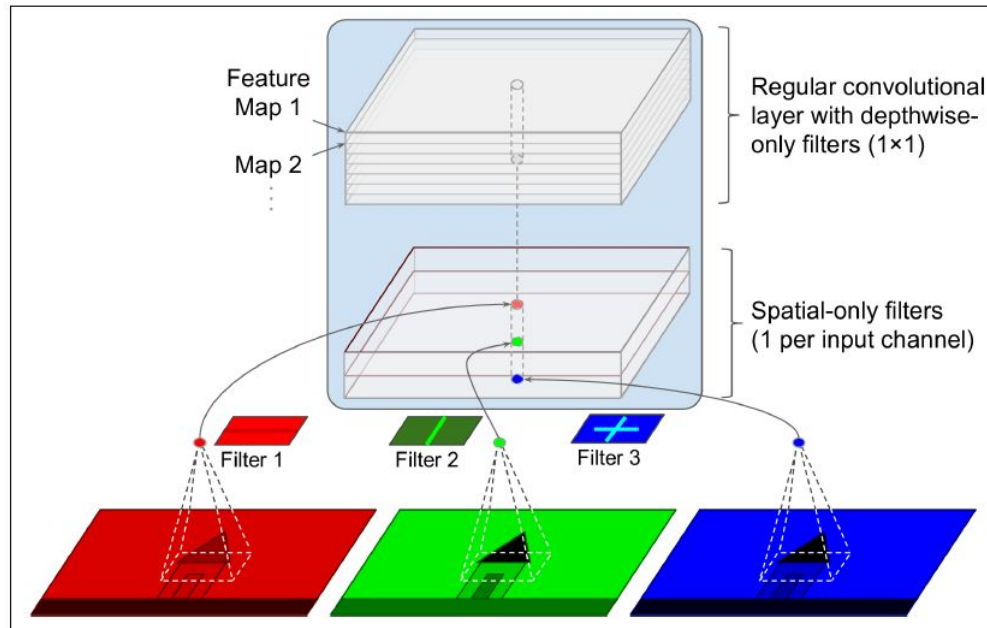


Figure 14-19. Depthwise Separable Convolutional Layer

SENet (1) : 2017

Squeeze-and-Excitation Network (SENet): 2.25% top-5 error rate.

Extended versions of inception networks and ResNet are called SE-Inception and SE-ResNet respectively. The boost comes as a SENet adds a small neural network, called a SE Block, to every unit in the original architecture (i.e., every inception module or every residual unit).

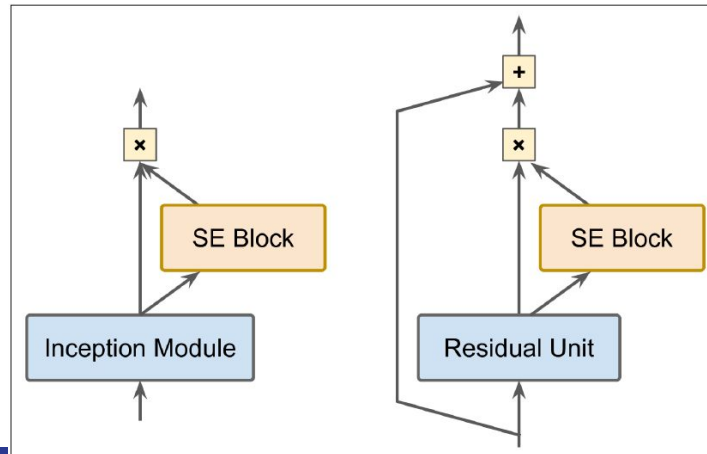


Figure 14-20. SE-Inception Module (left) and SE-ResNet Unit (right)

SENet (2)

SE Block analyzes the output of the unit it is attached to, focusing exclusively on the depth dimension (it does not look for any spatial pattern), and it learns which features are usually most active together. (if you see a mouth and a nose, you should expect to see eyes as well.)

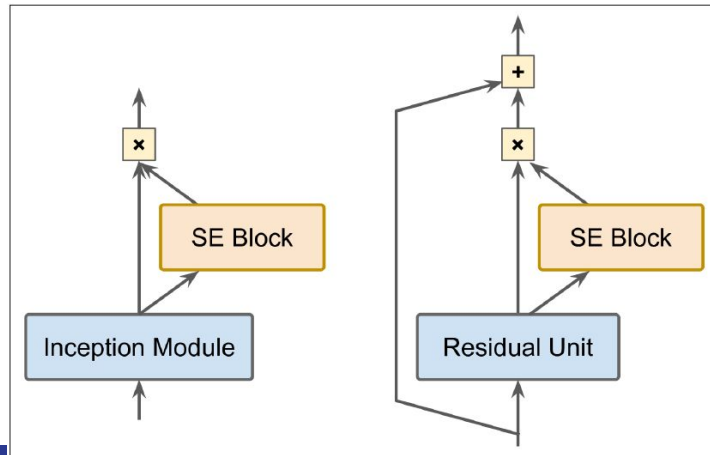


Figure 14-20. SE-Inception Module (left) and SE-ResNet Unit (right)

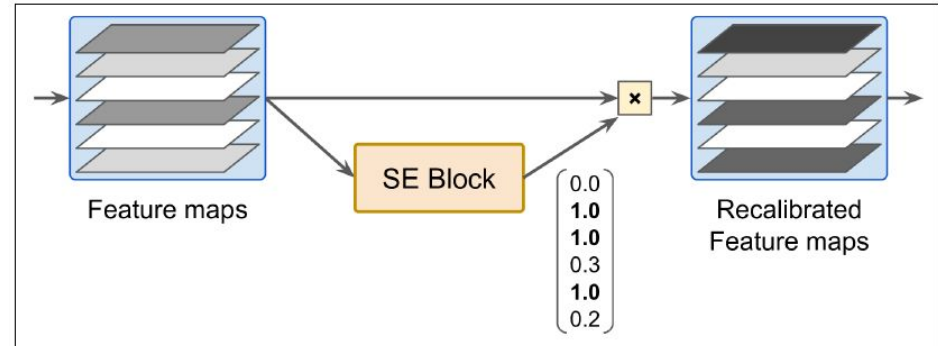


Figure 14-21. An SE Block Performs Feature Map Recalibration

SENet (3)

A SE Block is composed of just 3 layers: a global average pooling layer, a hidden dense layer using the ReLU activation function, and a dense output layer using the sigmoid activation function.

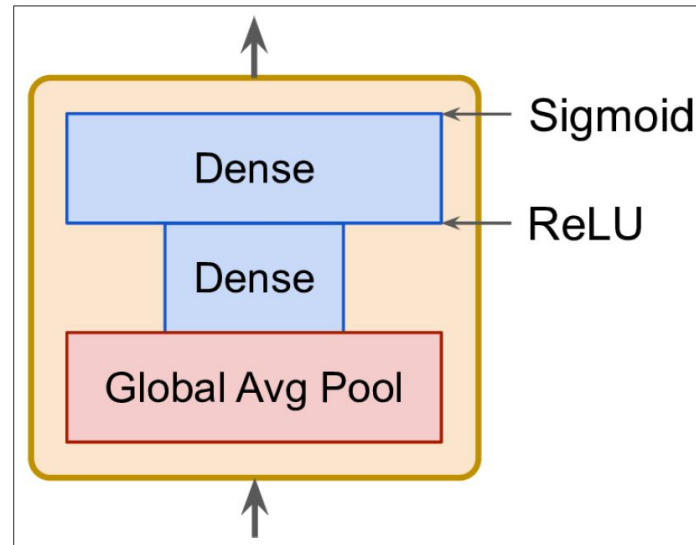


Figure 14-22. SE Block Architecture

Object Detection

Object detection: classifying and localizing multiple objects in an image.

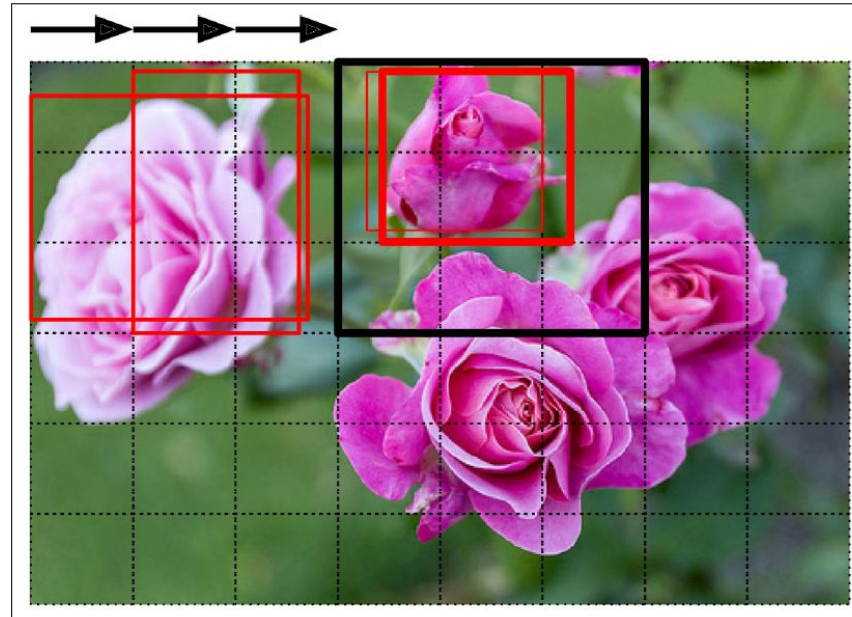
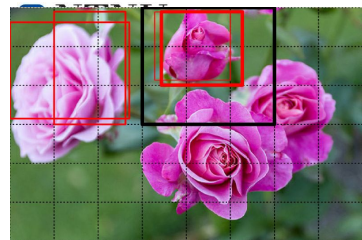


Figure 14-24. Detecting Multiple Objects by Sliding a CNN Across the Image

Object Detection : non-max suppression



1. Add an extra *objectness* output to your CNN, to estimate the probability that a flower is indeed present in the image (alternatively, add a “no-flower” class). Use the sigmoid activation function and the “binary_crossentropy” loss. Get rid of all the bounding boxes for which the objectness score is below some threshold: this will drop all the bounding boxes that don’t actually contain a flower.
2. Find the bounding box with the highest objectness score, and get rid of all the other bounding boxes that overlap a lot with it. (The bounding box with the max objectness score is the thick bounding box over the top-most rose). The other bounding box over that same rose overlaps a lot with the max bounding box, so we will get rid of it. (Repeat step two until there are no more bounding boxes to get rid of.)

Object Detection : Fully Convolutional Networks (FCNs)

Semantic segmentation : classifying every pixel in an image according to the class of the object it belongs to.

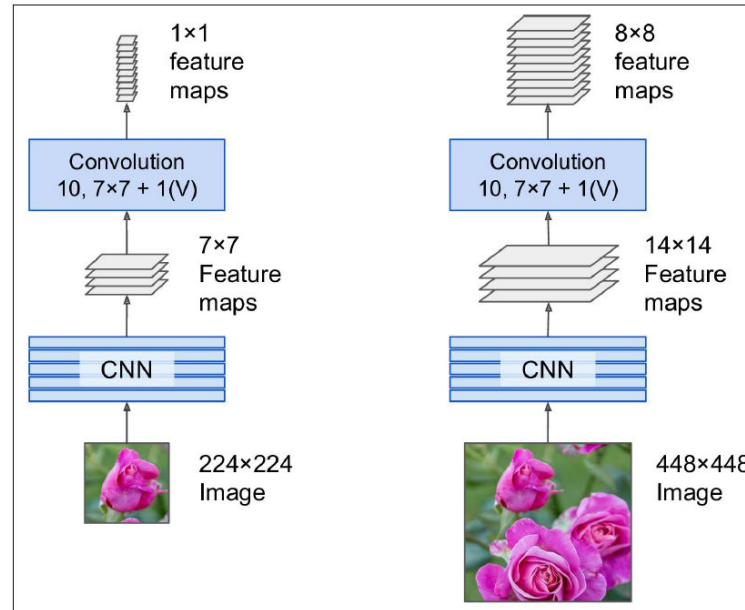


Figure 14-25. A Fully Convolutional Network Processing a Small Image (left) and a Large One (right)

You Only Look Once (YOLO)

Proposed in 2015 (v1). Currently up to v11. Uses one forward propagation through the neural network.

Semantic Segmentation



Figure 14-26. Semantic segmentation

Takeaways

- Pooling layer definition and role.
- CNN Architectures: evolution and improvements.